DenseNet121 - Kod Açıklama Raporu

DenseNet121 Ağız ROI Deepfake Tespiti - Kod Açıklama Raporu

Notebook mimarisi, veri hattı, kalite kapıları, eğitim, değerlendirme ve kurtarma düzeninin ayrıntılı açıklaması

Belge kapsamı

Notebook'taki 15 ana hücre ile çalışma zamanı kopması sonrasında kullanılan kurtarma ve dosya düzeltme hücrelerini teknik olarak açıklar. Kodun neden bu sırayla kurulduğunu, hangi dosyaları ürettiğini ve her kalite kontrolünün amacını gösterir.

Alan	Değer
Run ID	20260808_0856_mouth_densenet121_seed42
Notebook	DenseNet121_Mouth_Deepfake_Colab_FIXED.ipynb
Model	ImageNet ön eğitimli DenseNet121
Veri	2.987 ağız ROI: 2.389 train, 296 validation, 302 test
Hazırlayan	Dilara Öztürk
Tarih	8 Ağustos 2026

Bu rapor, yüklenen notebook ve doğrulanmış deney ZIP paketindeki gerçek konfigürasyon, log, metrik, tahmin ve grafiklerden hazırlanmıştır.

1. Kod tasarımının genel yapısı

Kod; ortam kurulumu, veri keşfi, metadata denetimi, TensorFlow veri hattı, model kurulumu, smoke test, iki aşamalı eğitim, bağımsız test ve final paketleme olarak ayrılmıştır. Bu sıralama, eğitim başlamadan önce veri ve sızıntı sorunlarını görünür hâle getirir; test kümesini model seçiminden izole eder; bağlantı kopmalarında kaybı sınırlar.

Aşama	Hücreler	Ana çıktı
Ortam ve depolama	1-2	GPU, seed, local source ve deney kökleri
Veri keşfi	3-5	2.987 ağız ROI ve tf.data hattı
Metadata ve sızıntı	6-9	Manifest, SHA-256 ve kesişim raporu
Model ve test kapıları	10-11	DenseNet121, smoke ve checkpoint testi
Eğitim	12-13	Frozen + fine-tuning, best/last model
Değerlendirme	14	Validation eşiği, test metrikleri ve grafikler
Kurtarma ve paketleme	Kurtarma + 15	Temiz yükleme, audit, checksum ve ZIP

Tek model ilkesi

Bu notebook'ta HOG, LBP, KAZE, SVM veya başka bir sınıflandırıcı kullanılmaz. DenseNet121 gövdesi ile sigmoid sınıflandırma başlığı birlikte tek ana modeldir.

1.1. Çıktı klasörü yaklaşımı

Her deney, YYYYMMDD_HHMM_mouth_densenet121_seed42 biçimindeki Run ID ile ayrı klasöre yazılır. Checkpoint, log, metrik, tahmin, figür ve metadata birbirinden ayrılır. Bu düzen eski sonuçların üzerine yazılmasını engeller ve rapor üretimini kolaylaştırır.

2. Hücre 1-3: ortam, local source ve veri keşfi

Hücre 1, Python/TensorFlow/NumPy sürümlerini yazdırır, Python, NumPy ve TensorFlow için seed=42 uygular ve T4 GPU'yu doğrular. GPU memory growth ayarı, TensorFlow'un GPU belleğini ihtiyaç oldukça kullanmasını sağlar.

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Hücre 2, Türkçe klasör adlarının Unicode NFC/NFD farklarından etkilenmemek için adları normalize ederek local source klasörlerini tek tek bulur. Böylece 'Çalışmalar' ve 'Sonuçlar' gibi adların farklı kodlanması
doğrudan yol hatasına dönüşmez.

Hücre 3, Ağız klasörünü salt okunur biçimde tarar. Klasör ağacını ve doğrudan görüntü sayılarını gösterir; mouth, debug ve landmarks ayrımını görünür kılar.

Split	Real	Fake	Toplam
Train	1.197	1.192	2.389
Validation	155	141	296
Test	146	156	302
Toplam	1.498	1.489	2.987

Önemli seçim

Model girdisi olarak yalnızca mouth PNG dosyaları kullanılır; debug görüntüleri ve landmark JSON dosyaları eğitim örneği sayılmaz.

3. Hücre 4-5: kayıtlar ve TensorFlow veri hattı

Hücre 4, real=0 ve fake=1 etiketlemesini kurar; klasör başına sayıları beklenen değerlerle assert eder; split yollarının birbirinden ayrık olduğunu kontrol eder. Run klasörleri ve başlangıç konfigürasyonu bu noktada oluşturulur.

Hücre 5, dosya yollarından görüntüleri tembel biçimde okuyan bir tf.data.Dataset hattı kurar. Görüntüler 224x224 RGB biçimine getirilir ve DenseNet'in preprocess_input dönüşümünden geçirilir. Batch boyutu 16'dır; prefetch GPU'nun veri bekleme süresini azaltır.

Dönüşüm	Yalnız train?	Gerekçe
Horizontal flip	Evet	Sağ-sol değişimine dayanıklılık
Rotation ±%3	Evet	Küçük poz/kırpma değişimleri
Zoom ±%8	Evet	ROI ölçek farkları
Translation ±%4	Evet	Kırpma merkezi sapmaları
Contrast ±%10	Evet	Aydınlatma ve sıkıştırma farkları
DenseNet preprocessing	Tümü	ImageNet ağırlıklarıyla giriş uyumu

Validation ve test kümelerinde rastgele augmentation yoktur. Bu izolasyon, aynı test örneğinin farklı rastgele görünümlerle değerlendirilmesi gibi protokol belirsizliklerini engeller.
Smoke çıktısı
150 train, 19 validation ve 19 test batch'i; örnek batch şekli (16, 224, 224, 3). Veri hattı model kurulmadan önce doğrulanmıştır.

4. Hücre 6-9: metadata, muhasebe ve sızıntı
denetimi
Hücre 6 dosya adlarını inceler; Hücre 7 mevcut CSV/JSON/NPZ manifestlerini arar; Hücre 8 ana metadata dosyasını okur. Ana metadata 3.098 kayıt ve 24 sütundan oluşur. Kayıtlar 2.987 SUCCESS, 111 SKIPPED ve 0 ERROR olarak ayrılır.
Muhasebe eşitliği
3.098 metadata kaydı = 2.987 başarılı ağız ROI + 111 atlanan kayıt + 0 hata.
Hücre 9, başarılı metadata kayıtlarını diskteki gerçek PNG dosyalarıyla dosya adı üzerinden eşleştirir. Her örnek için kararlı sample_id, source_video, frame_index, face_index, label, split, SHA-256 ve güncel output_path içeren model manifesti üretir.
Kontrol
Train-Val
Train-Test
Val-Test
source_video kesişimi
0
0
0
SHA-256 içerik kesişimi
0
0
0
Kaynak kimliği kesişiminin sıfır olması aynı kaynak kaydın farklı splitlere düşmediğini; SHA-256 kesişiminin sıfır olması ise byte düzeyinde aynı görüntünün splitler arasında tekrar etmediğini gösterir. İki kontrol birbirini tamamlar.
Standart Run ID, bu hücrede 20260808_0856_mouth_densenet121_seed42 olarak kesinleştirilir ve manifest/raporlar atomik yazılır.

5. Hücre 10: DenseNet121 mimarisi
ImageNet ön eğitimli DenseNet121, include_top=False ile yüklenir. Hazır ImageNet sınıflandırma katmanı çıkarılır; ağız ROI'ye özel ikili başlık eklenir.
Katman
Çıktı/Görev
Eğitim durumu (Aşama 1)
Input
224x224x3 ağız ROI
-
DenseNet121
Uzamsal özellik haritaları
Donmuş
GlobalAveragePooling2D
1.024 boyutlu özet temsil
Parametresiz
Dropout 0,40
Aşırı öğrenmeyi azaltır
Aktif
Dense(1, sigmoid)
Fake olasılığı
Eğitilebilir
Mixed precision T4 üzerinde hesaplamayı hızlandırır; son sigmoid katmanı sayısal kararlılık için float32 tutulur. Kayıp Binary Cross-Entropy, optimizer Adam'dır. Keras metrikleri accuracy, precision, recall, ROC-AUC ve PR-AUC olarak tanımlanır.
Parametre grubu
Sayı
Toplam
7.038.529
Frozen aşamada eğitilebilir
1.025
Frozen aşamada eğitilemez

7.037.504
DenseNet gövdesi
7.037.504
Özellik çıkarımı
HOG/LBP gibi el yapımı tanımlayıcılar yoktur. Yoğun bağlantılı DenseNet blokları kenar, renk geçişi, dudak/diş dokusu ve daha birleşik ağız örüntülerini otomatik temsil eder.

6. Hücre 11: smoke, gradyan ve checkpoint testi
Tam eğitimden önce iki batch üzerinde ileri/geri yayılım gerçekleştirilir. Girdiler, etiketler, loss, tahminler ve tüm gradyanlar için NaN/Inf kontrolü yapılır. İki optimizer adımından sonra model geçici .keras dosyasına kaydedilir, yeniden yüklenir ve tahmin/loss eşitliği test edilir.
Test
Doğrulanan değer
İki batch train_on_batch
Optimizer sayacı 0'dan 2'ye çıktı
Gradyan testi
Loss 0.777588; 2 gradyan tensörü sonlu
Checkpoint yeniden yükleme
Önce/sonra loss 0.521516
Tahmin eşitliği
Maksimum mutlak fark 0.0
Gerçek eğitim hazırlığı
Başlangıç ağırlıkları geri yüklendi; optimizer sayacı 0
Bu yaklaşım smoke testte yapılan iki güncellemenin gerçek eğitimi kirletmesini engeller. Tam model dosyası mimari, ağırlıklar ve optimizer durumunu birlikte saklar. Geçici dosya doğrulandıktan sonra os.replace ile yayımlanır.
Kalite kapısı
Smoke test başarısızsa frozen eğitim başlatılmaz. Böylece uzun bir koşudan sonra fark edilecek veri, gradyan veya checkpoint hataları erkenden yakalanır.

7. Hücre 12-13: iki aşamalı eğitim
Frozen aşamada DenseNet gövdesi sabit, yalnızca sigmoid başlık eğitilebilir. En fazla 12 epoch, başlangıç öğrenme oranı 1e-3 kullanılır. Model seçimi validation ROC-AUC üzerinden yapılır; test kümesi kullanılmaz.
Aşama
Epoch
Öğrenme oranı
En iyi validation AUC
Frozen
1-12
1e-3 (sonda 5e-4)
0.7335 (epoch 9)
Fine-tuning
13-22
1e-5
0.7800 (epoch 21)
Fine-tuning aşamasında en iyi frozen checkpoint yüklenir. DenseNet'in yalnızca conv5_block1...conv5_block16 katmanları açılır; Batch Normalization katmanları küçük batch istatistiklerinin ön eğitim dağılımını bozmasını engellemek için donmuş tutulur.
AtomicModelCheckpoint her epoch sonunda last.keras kaydeder; val_auc iyileştiğinde best.keras günceller. BackupAndRestore, çalışma zamanı koparsa eğitim durumunu geri yüklemek için ayrı yedek üretir. CSVLogger, ReduceLROnPlateau, EarlyStopping, TerminateOnNaN ve FiniteMetricsGuard birlikte kullanılır.
Sonuç
Fine-tuning, en iyi validation AUC'yi 0,7335'ten 0,7800'a yükseltmiştir. Bu yaklaşık 0,0464 mutlak artış, ağız/diş temsillerine kontrollü uyarlamanın yararlı olduğunu gösterir.

8. Hücre 14: eşik seçimi ve bağımsız test
Önce validation olasılıkları üretilir. ROC eğrisindeki Youden J = TPR - FPR değerini en büyük yapan eşik 0,508568 olarak seçilir. Bu işlem sırasında test etiketleri kullanılmaz. Eşik sabitlendikten sonra test seti bir kez değerlendirilir.
Seçim değeri
Sonuç
Validation eşiği

0.508568
Youden J
0.456280
Validation ROC-AUC
0.779547
Validation F1
0.727273
Görüntü tahminleri manifest sırasıyla eşleştirilir ve örnek bazında CSV'ye yazılır. Aynı source_video altındaki birden fazla ROI'nin fake olasılığı ortalanarak 292 kaynak düzeyi tahmin üretilir. Confusion matrix, ROC/PR eğrileri, olasılık dağılımı ve birleşik eğitim eğrileri İngilizce etiketlerle yüksek çözünürlükte kaydedilir.
Test metrikleri accuracy, balanced accuracy, precision, recall, specificity, F1, ROC-AUC, PR-AUC ve Matthews korelasyon katsayısını içerir. Ayrıca 0,5 varsayılan eşik sonuçları şeffaf karşılaştırma için ayrı
dosyada tutulur.

9. Kurtarma hücresi ve Hücre 15
Eğitim ve test tamamlandıktan sonra Colab çalışma zamanı kopmuştur. Kurtarma hücresi modeli yeniden eğitmek yerine sabit Run ID'yi bulmuş, klasörleri ve manifesti yeniden yüklemiş, test veri hattını manifestten kurmuş ve best.keras üzerinde temiz tahmin yapmıştır.
Kurtarılan öğe
Doğrulama
Run
20260808_0856_mouth_densenet121_seed42
Best model
Diskten yüklendi ve sonlu olasılık üretti
Manifest
2.987 satır
Test veri hattı
302 kayıt
Global best
Epoch 21; val AUC 0,779959
Hücre 15 gerekli klasör/dosyaları denetler; kaydedilmiş tabloları yeniden okur; best.keras dosyasını temiz biçimde yükleyerek inference testi yapar; tüm figürlerin kısa kenarının en az 600 px olduğunu doğrular. Sonra bağımlılık sürümleri, insan okunur RUN_SUMMARY, final_audit ve tüm artefaktlar için SHA-256 manifesti oluşturulur.
Final ZIP geçici adla üretilir, zipfile.testzip ile bütünlük kontrolünden geçirilir ve atomik olarak yayımlanır. Paket 204,62 MB, 47 üye ve bağımsız ZIP SHA-256 kaydına sahiptir.
Gerçek yaşam dayanıklılığı
Bağlantı kopması model veya metrik kaybına yol açmamıştır. best/last checkpoint, backup, manifest ve Run ID tasarımı sayesinde yalnızca finalizasyon belleği yeniden kurulmuştur.

10. Üretilen dosyalar ve teknik yorum
Klasör/Dosya
İçerik
checkpoints/best.keras
Validation AUC'ye göre global en iyi tam model
checkpoints/last.keras
Son tamamlanan epoch modeli
checkpoints/*backup
Kesinti sonrası devam ağırlıkları ve metadata
metadata/model_manifest.csv
2.987 örnek, kaynak, split, hash ve yol
metrics/*history.csv
Frozen, fine-tuning ve birleşik epoch kayıtları
metrics/final_*metrics.csv
Görüntü ve kaynak düzeyi sonuçlar
predictions/*.csv
302 görüntü ve 292 kaynak tahmini
figures/*.png
Eğitim, confusion, ROC/PR, olasılık grafikleri
artifact_manifest.csv
Artefakt boyutu ve SHA-256 listesi
final_audit.json
Final kalite kapıları ve sonuç özeti
Kod modüler bir notebook biçiminde olsa da üretim ortamına taşınırken veri, model, eğitim ve

değerlendirme fonksiyonları ayrı Python modüllerine ayrılabilir. Bununla birlikte mevcut deney; konfigürasyonun çözülmesi, veri muhasebesi, test izolasyonu, checkpoint ve çıktı doğrulama bakımından yeniden üretilebilir bir araştırma koşusu sunmaktadır.

11. Kaynakça

[1] Elhassan, A., Al-Fawa'reh, M., Jafar, M. T., Ababneh, M., & Jafar, S. T. (2022). DFT-MF: Enhanced deepfake detection
using mouth movement and transfer learning. SoftwareX, 19, 101115.
https://doi.org/10.1016/j.softx.2022.101115

[2] Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks.
CVPR, 4700-4708. https://arxiv.org/abs/1608.06993

[3] Keras Applications. DenseNet API and preprocess_input. https://keras.io/api/applications/densenet/